

Guia e roteiro de estudos

Revisão da Primeira Prova Bimestral

Conteúdo autoral derivado do lesson model aprovado; a bibliografia é complementar.

Razões para estudar conceitos de LP

O estudo dos conceitos de linguagens de programação amplia a expressão de ideias, fundamenta a escolha de linguagens, facilita a aprendizagem de novas linguagens, melhora o uso das já conhecidas e ajuda a compreender a implementação.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01, obj13.

Fontes: slides-ch1, slide 3, paraphrase

Crítérios de Avaliação de Linguagem

Legibilidade, capacidade de escrita, confiabilidade e custo são critérios usados para comparar a adequação de uma linguagem, sem pressupor que uma única linguagem maximize todos eles.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01, obj13.

Fontes: slides-ch1, slide 5, paraphrase

Legibilidade e ortogonalidade

Legibilidade envolve a facilidade de compreender programas. A ortogonalidade contribui para ela ao permitir que componentes básicos sejam combinados com poucas restrições e exceções.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01.

Fontes: slides-ch1, slide 7, paraphrase

Capacidade de escrita

É a facilidade com que uma linguagem pode ser usada para criar programas destinados a um domínio de problema específico.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01.

Fontes: slides-ch1, slide 5, paraphrase

Confiabilidade e verificação de tipos

Confiabilidade é o comportamento de acordo com as especificações. A verificação de tipos contribui para esse critério ao detectar usos incompatíveis, preferencialmente durante a compilação para reduzir o custo de correção.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01, obj13.

Fontes: slides-ch1, slide 14, paraphrase

Custo de uma linguagem

O custo inclui treinamento, aprendizagem, escrita de programas, implementação e execução. Decisões como verificações em tempo de execução podem elevar a confiabilidade, mas também aumentar o custo de execução.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01, obj02.

Fontes: slides-ch1, slide 18, paraphrase

Influências no Projeto de Linguagens

O projeto de linguagens é influenciado pela arquitetura de computadores e pelas metodologias de projeto de software. A Arquitetura de von Neumann favoreceu o modelo imperativo, enquanto novas metodologias contribuíram para novos paradigmas.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj02, obj13.

Fontes: slides-ch1, slide 21, paraphrase; slides-ch1, slide 22, paraphrase

Categorias de Linguagens

As categorias apresentadas são imperativas, funcionais, lógicas e híbridas de marcação-programação. Elas diferem pela centralidade de variáveis e atribuições, aplicação de funções, regras declarativas ou combinação de marcação com ações de programação.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj02, obj04.

Fontes: slides-ch1, slide 26, paraphrase

Trade-offs no Projeto de Linguagens

Uma decisão de projeto pode beneficiar um critério e prejudicar outro. Os contrastes destacados são confiabilidade versus custo de execução, legibilidade versus capacidade de escrita e flexibilidade versus confiabilidade.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj01, obj02, obj13.

Fontes: slides-ch1, slide 27, paraphrase

Métodos de Implementação e Ambientes de Programação

Compilação traduz para linguagem de máquina; interpretação pura executa por meio de um interpretador; sistemas híbridos usam código intermediário; e JIT compila durante a execução. Ambientes de programação reúnem ferramentas de desenvolvimento.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj02, obj08, obj13.

Fontes: slides-ch1, slide 28, paraphrase; slides-ch1, slide 36, paraphrase; slides-ch1, slide 38, paraphrase

Evolução das principais linguagens de programação

A evolução deve ser estudada pela relação entre o ambiente de criação, as motivações, as contribuições e a influência sobre linguagens posteriores, e não como uma sequência em que cada linguagem simplesmente substitui a anterior.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj13.

Fontes: sebesta-11ed, p. 50, paraphrase; exercises-ch2, p. 1, inference

Plankalkül

Plankalkül foi concebida por Konrad Zuse para expressar computações, mas não foi implementada em sua época e teve sua descrição publicada muito depois. Sua relevância histórica decorre de antecipar recursos avançados, especialmente em estruturas de dados.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04.

Fontes: sebesta-11ed, p. 52, paraphrase; exercises-ch2, p. 1, teacher_provided

Pseudocódigos e interpretação inicial

Sistemas iniciais como Short Code e Speedcoding simplificaram a programação por meio de interpretação e operações de nível mais alto, mas pagaram por isso com maior tempo de execução e limitações de recursos.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04.

Fontes: sebesta-11ed, p. 55, paraphrase; exercises-ch2, p. 1, teacher_provided

Fortran e computação científica

A evolução de Fortran incorporou recursos como declarações de tipo explícitas, estruturas de controle, passagem de subprogramas, vetores dinâmicos, registros, ponteiros, módulos e recursão, acompanhando novas necessidades de programação científica.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04.

Fontes: sebesta-11ed, p. 59, paraphrase; exercises-ch2, p. 1, teacher_provided

Lisp, Scheme e programação funcional

A linhagem de Lisp favorece computação baseada em funções e listas. Scheme destaca tamanho reduzido, escopo estático e funções como entidades de primeira classe, que podem ser atribuídas, passadas e retornadas.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04, obj12.

Fontes: sebesta-11ed, p. 65, paraphrase

Linguagens orientadas por domínio

COBOL e Basic ilustram projetos guiados por públicos e domínios distintos: processamento comercial com notação próxima da linguagem natural e ampliação do acesso à programação para estudantes não especializados.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04.

Fontes: sebesta-11ed, p. 73, paraphrase; sebesta-11ed, p. 77, paraphrase

Linguagens dinâmicas: APL e SNOBOL

APL e SNOBOL apresentam tipagem dinâmica e alocação dinâmica de armazenamento: a variável adquire tipo e armazenamento quando recebe um valor.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04, obj11.

Fontes: sebesta-11ed, p. 85, paraphrase; exercises-ch2, p. 2, teacher_provided

Linhagens imperativa, lógica e orientada a objetos

A revisão histórica contrasta linhagens baseadas em comandos e estado, programação baseada em regras e linguagens orientadas a objetos. Ada relaciona recursos de linguagem a sistemas críticos, e Smalltalk ocupa posição central no suporte completo à orientação a objetos.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04, obj13.

Fontes: sebesta-11ed, p. 92, paraphrase; sebesta-11ed, p. 98, paraphrase; exercises-ch2, p. 2, teacher_provided

Java, Lua e estratégias de implementação

Java representa a combinação do paradigma imperativo com orientação a objetos e um sistema de execução amplamente disponível. Lua combina tipagem dinâmica, tabelas, funções de primeira classe, coleta de lixo e tradução para código intermediário interpretado.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04, obj02.

Fontes: sebesta-11ed, p. 106, paraphrase; sebesta-11ed, p. 113, paraphrase; slides-ch1, slide 34, paraphrase

Linguagens híbridas de marcação-programação

Uma linguagem híbrida de marcação-programação estende marcação com elementos capazes de especificar ações como controle de fluxo e computação. XSLT e JSP aparecem como exemplos históricos dessa categoria.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj03, obj04.

Fontes: sebesta-11ed, p. 116, paraphrase; exercises-ch2, p. 3, teacher_provided

Sintaxe e Semântica

Sintaxe é a forma ou estrutura das expressões, atribuições e unidades de programa; semântica é o significado dessas construções. Juntas, caracterizam a definição de uma linguagem de programação.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj05, obj13.

Fontes: slides-ch3, slide 3, paraphrase

Sentença, linguagem, lexema e token

Sentença é uma cadeia de caracteres sobre um alfabeto; linguagem é um conjunto de sentenças; lexema é uma unidade sintática elementar; e token é uma categoria que reúne lexemas.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj05, obj08.

Fontes: slides-ch3, slide 4, paraphrase

Reconhecedores e Geradores

Um reconhecedor decide se uma cadeia pertence à linguagem. Um gerador descreve ou produz as sentenças da linguagem e oferece uma estrutura com a qual uma sentença pode ser comparada.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj05, obj06, obj08.

Fontes: slides-ch3, slide 6, paraphrase

FNB e Gramáticas Livres de Contexto

Gramáticas Livres de Contexto são geradores usados para descrever classes de linguagens. A Forma Normal de Backus foi criada para descrever a sintaxe de Algol 58 e é equivalente às gramáticas livres de contexto.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06.

Fontes: slides-ch3, slide 7, paraphrase

Terminais, não terminais e regras

Na FNB, não terminais representam classes de estruturas sintáticas, enquanto terminais correspondem a tokens ou lexemas. Cada regra tem um não terminal à esquerda e terminais e/ou não terminais à direita.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06.

Fontes: slides-ch3, slide 8, paraphrase

Recursão, derivação e forma sentencial

Recursão permite descrever listas sintáticas. Derivação é a aplicação repetida de regras desde o símbolo inicial; cada cadeia intermediária é uma forma sentencial, e uma forma composta apenas por terminais é uma sentença.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06, obj09.

Fontes: slides-ch3, slide 11, paraphrase; slides-ch3, slide 14, paraphrase

Árvore de Análise Sintática

É a representação hierárquica de uma derivação, mostrando como símbolos não terminais são expandidos até produzir os terminais da sentença.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06, obj08, obj09.

Fontes: slides-ch3, slide 15, paraphrase

Ambiguidade em Gramáticas

Uma gramática é ambígua quando uma mesma sentença pode produzir duas árvores de derivação diferentes. A organização das regras pode expressar precedência de operadores e eliminar uma ambiguidade.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06, obj07, obj09.

Fontes: slides-ch3, slide 16, paraphrase; slides-ch3, slide 18, paraphrase

FNB Extendida

A FNBE usa colchetes para termos opcionais, parênteses e barras para alternativas e chaves para repetições de zero ou mais termos.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06, obj09.

Fontes: slides-ch3, slide 21, paraphrase

Semântica Estática e Semântica Dinâmica

A semântica estática trata restrições que uma GLC não descreve adequadamente e pode ser especificada por gramáticas de atributos. A semântica dinâmica pode ser descrita pelos métodos operacional, denotacional e axiomático.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj07, obj13.

Fontes: slides-ch3, slide 24, paraphrase; slides-ch3, slide 25, paraphrase; slides-ch3, slide 34, paraphrase; slides-ch3, slide 39, paraphrase; slides-ch3, slide 49, paraphrase

Análise Léxica e Análise Sintática

O processamento sintático é dividido entre o analisador léxico, baseado em padrões e gramática regular, e o parser, baseado em GLC ou FNB e modelado como autômato com pilha.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj08, obj13.

Fontes: slides-ch4, slide 4, paraphrase

Separação entre léxico e parser

Separar análise léxica e análise sintática favorece simplicidade, permite otimizações específicas do léxico e isola aspectos menos portáteis.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj08.

Fontes: slides-ch4, slide 6, paraphrase

Casamento de padrões e geração de tokens

O analisador léxico encontra subcadeias no código-fonte, agrupa lexemas e atribui códigos internos. Um token pode representar vários lexemas, como diferentes nomes classificados como IDENT.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj05, obj08.

Fontes: slides-ch4, slide 7, paraphrase

Implementação do analisador léxico

O analisador léxico pode ser construído com ferramentas a partir de descrições formais, com código baseado em diagramas de estado ou com tabelas montadas manualmente. Funções como getChar, addChar e lookup apoiam o reconhecimento.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj08.

Fontes: slides-ch4, slide 9, paraphrase; slides-ch4, slide 15, paraphrase

Objetivos da análise sintática

O parser deve localizar erros sintáticos, produzir diagnósticos, recuperar-se para continuar a análise e construir a árvore de análise sintática ou um caminho correspondente.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj08, obj09.

Fontes: slides-ch4, slide 18, paraphrase

Análise Sintática Descendente

A análise descendente constrói a árvore a partir da raiz, usa derivação mais à esquerda e pode ser implementada por uma coleção de subprogramas, muitos deles recursivos.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj09.

Fontes: slides-ch4, slide 19, paraphrase; slides-ch4, slide 26, paraphrase

Lookahead

Quando um não terminal possui alternativas, o analisador descendente compara o próximo token com os primeiros tokens que cada regra pode gerar. A ausência de correspondência indica erro sintático.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj09.

Fontes: slides-ch4, slide 31, paraphrase

Recursão à esquerda

Gramáticas com recursão à esquerda direta ou indireta não podem ser usadas diretamente por um analisador descendente. A gramática deve ser transformada para substituir essa recursão por uma forma adequada.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj06, obj09.

Fontes: slides-ch4, slide 36, paraphrase

Análise Sintática Ascendente e redução

A análise ascendente constrói a árvore a partir das folhas e procura uma subcadeia que corresponda ao lado direito de uma regra, reduzindo-a para recuperar a forma sentencial anterior da derivação mais à direita.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj09, obj10.

Fontes: slides-ch4, slide 22, paraphrase; slides-ch4, slide 42, paraphrase

Analísadores LR, ACTION e GOTO

Analísadores LR são analisadores ascendentes normalmente construídos com ferramentas. A tabela ACTION determina ações com base no estado e no próximo terminal; GOTO determina o novo estado depois de uma redução usando não terminais.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj10.

Fontes: slides-ch4, slide 49, paraphrase; slides-ch4, slide 51, paraphrase

Nomes e palavras especiais

O projeto de nomes considera distinção entre maiúsculas e minúsculas e tratamento das palavras especiais. Palavra-chave é especial em certos contextos; palavra reservada não pode ser usada como nome.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj13.

Fontes: slides-ch5, slide 4, paraphrase; slides-ch5, slide 8, paraphrase

Variável e seus atributos

Uma variável é uma abstração de uma célula de memória e pode ser caracterizada por nome, endereço, valor, tipo, tempo de vida e escopo.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11.

Fontes: slides-ch5, slide 9, paraphrase

Endereço e apelidos

O endereço é a posição de memória associada à variável. Quando dois nomes acessam a mesma posição, eles são apelidos; isso pode prejudicar a legibilidade porque o programador precisa acompanhar referências alternativas.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj01.

Fontes: slides-ch5, slide 11, paraphrase

Vinculação e tempo de vinculação

Vinculação é uma associação entre atributo e entidade, como variável e tipo ou operação e símbolo. Tempo de vinculação é o momento em que a associação ocorre.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj13.

Fontes: slides-ch5, slide 13, paraphrase; slides-ch5, slide 14, paraphrase

Vinculação estática e dinâmica

A vinculação estática ocorre antes da execução e permanece inalterada; a dinâmica ocorre pela primeira vez durante a execução ou pode mudar enquanto o programa executa.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11.

Fontes: slides-ch5, slide 17, paraphrase

Declaração e vinculação dinâmica de tipos

Declaração explícita associa nomes a tipos por uma sentença; declaração implícita usa convenções. Na vinculação dinâmica de tipos, atribuições podem alterar o tipo associado, aumentando flexibilidade, mas dificultando a detecção antecipada de erros.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj01.

Fontes: slides-ch5, slide 19, paraphrase; slides-ch5, slide 21, paraphrase

Alocação, desalocação e tempo de vida

Alocação obtém uma célula de memória e desalocação a devolve. O tempo de vida de uma variável é o período em que ela permanece vinculada a uma posição específica de memória.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj12.

Fontes: slides-ch5, slide 22, paraphrase

Categorias de variáveis por tempo de vida

As categorias apresentadas são estáticas, dinâmicas da pilha, dinâmicas de heap e dinâmicas de heap implícito. Elas diferem pelo momento e mecanismo de alocação, flexibilidade, eficiência, suporte à recursão e possibilidade de detecção de erros.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj13.

Fontes: slides-ch5, slide 23, paraphrase; slides-ch5, slide 24, paraphrase; slides-ch5, slide 25, paraphrase; slides-ch5, slide 26, paraphrase

Escopo estático, escopo dinâmico e ambiente de referenciamento

Escopo é a faixa de sentenças em que uma variável é visível. No escopo estático, a declaração é encontrada pela organização textual e pelos ancestrais estáticos; no dinâmico, pela sequência de chamadas. O ambiente de referenciamento reúne as variáveis visíveis em uma sentença.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj12, obj13.

Fontes: slides-ch5, slide 27, paraphrase; slides-ch5, slide 28, paraphrase; slides-ch5, slide 38, paraphrase; slides-ch5, slide 42, paraphrase

Escopo, tempo de vida e constantes nomeadas

Escopo e tempo de vida são conceitos diferentes: um trata visibilidade textual e o outro vinculação à memória. Uma constante nomeada recebe uma única vinculação de valor e favorece legibilidade, modificabilidade e parametrização.

Como estudar: explique o conceito sem consultar o texto, associe-o a um exemplo e compare-o com outro conceito ligado a obj11, obj12.

Fontes: slides-ch5, slide 41, paraphrase; slides-ch5, slide 45, paraphrase

Roteiro de revisão

1. Releia os objetivos. 2. Recupere cada definição de memória. 3. Compare os critérios. 4. Resolva os exercícios sem consulta. 5. Revise os pontos em que o feedback indicou dúvida.

As formulações são autorais e não reproduzem exercícios da bibliografia.